



BILDUNG. FREUDE INKLUSIVE.

Webdesign mit HTML und CSS

CSS-Animationen

Überarbeitet: 31.10.2024, Dipl.-Ing(FH) Robert Schoblick, M.Sc.



Kärntner Berufsförderungsinstitut GmbH
Bahnhofstraße 44
9020 Klagenfurt am Wörthersee
T. +43 (0)5 78 78
info@bfi-kaernten.at

www.bfi-kaernten.at



Die beste
Zeit für
Weiterbildung
ist JETZT!

Voraussetzung zum Verständnis:

- Grundkenntnisse in HTML und CSS
- Kenntnis der wichtigsten CSS-Eigenschaften (Farbgebung, Box-Modell etc.)
- Kenntnis der CSS-Eigenschaft „position“ (insbesondere relative und absolute Positionierung)

Arbeitsauftrag:

1. LESEN Sie dieses Skript aufmerksam durch!
2. Probieren Sie die in diesem Skript gezeigten Beispiele selbst aus. Benutzen Sie dafür wie üblich einen Texteditor – z.B.: Visual Studio Code, Notepad++ - und einen Webbrowser.
3. Animieren Sie einzelne Objekte (z.B.: Bilder, Menü-Button etc.) in einer optischen Betonung (zum Beispiel bei Mauszeiger-Überfahrt oder in zeitlichen Intervallen).
4. Programmieren Sie eine Laufschrift (Bitte nicht das veraltete HTML-Element <marquee> verwenden!), welche einen Text sowohl von rechts nach links als auch – in einem weiteren Beispiel – von links nach rechts in das Fenster schiebt.
5. Programmieren Sie einen Image-Slider (ohne JavaScript!) mithilfe von CSS-Animationen.
6. Programmieren Sie eine Webseite mit folgenden Inhalten:
 - * Ein „Klick me“-Button (gerne etwas größer), der unter einem Header in der Seite angeordnet wird (Der übrige Inhalt der Seite ist unerheblich! Dummy-Bilder und Fülltexte genügen!)
 - * Nach einer Sekunde soll über den Button ein Bild erscheinen (zuvor: scale: 0%) und den Button nach unten verschieben. Das Bild soll mit einem Link auf die bfi-Seite verknüpft sein.
7. Diskutieren Sie in einer kurzen Präsentation:
 - * Eventuell bekannte Beispiele in der Praxis für derartiges „Benehmen“
 - * Rechtslage und Grenzen der Rechtslage (auch in weiterer Folge: Datenschutz durch Cookie-Aktivierung)
 - * Ethische und moralische Aspekte.

***Die Code-Beispiele sind als Grafik in dieses Dokument eingefügt.
D.h.: Copy&Paste ist keine mögliche Lösung!***

Arbeiten Sie NICHT mit ChatGPT oder anderen KI-Systemen!!!!

„Lebendige“ Webseiten ohne JavaScript?

Um auf einer Webseite „Bewegung“ zu erzeugen, bedarf es nicht zwingend des Einsatzes von JavaScript. Moderne CSS3-Definitionen beherrschen „Animationen“. Darunter sind Änderungen von CSS-Eigenschaften bzw. derer Werte während der Darstellung der Webseite im Browser zu verstehen. Beispielsweise kann ein Bild langsam in die Seite einschweben, sich drehen oder die Größe ändern. Tatsächlich lassen sich mithilfe von CSS-Animationen sehr spannende und lebendige Effekte in der Webseite realisieren.

Animationen in CSS werden beinahe wie andere CSS-Regeln programmiert. Auch hier stellen Selektoren die Referenz auf die HTML-Elemente dar. Es wird allerdings zusätzlich eine „CSS-@-Regel“ benötigt, welche u.a. die zeitliche Verteilung einer Animation (innerhalb der vorgesehenen Dauer der Animation insgesamt) sowie die Änderungen der Eigenschaften selbst vorsieht: @keyframes.

„animation“ und „@keyframes“

Aufgabe: Eine Kreisfläche in der Webseite soll seine Farbe von hellgrau auf rot in einem Zeitraum von 10 Sekunden verändern! Dies soll allein durch CSS erreicht werden.

1. Schritt: HTML- und grundlegender CSS-Code:

Grundlage des Beispiels ist ein einfacher HTML- Code, der in eine Datei (zum Beispiel) index.html geschrieben wird. Der Code enthält bereits im „**head**“ einen CSS-Block (Element „**style**“), welcher rot hervorgehoben ist. Dieser CSS-Code legt die Höhe und die Breite eines **div**-Containers mit jeweils 100px sowie dessen Hintergrundfarbe mit hellgrau fest. Die Eigenschaft „**border-radius**“ rundet die Ecken des Elements mit einem Radius von 50px ab. Da diese 50px genau die Hälfte der Kantenlänge des Elements selbst entsprechen, erscheint die hellgraue Fläche wie ein Kreis im Webbrowser.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Animation</title>
  <style>
    div {
      width: 100px;
      height: 100px;
      border-radius: 50px;
      background-color: lightgray;
      position: absolute;
    }
  </style>
</head>
<body>
  <div></div>
</body>
</html>
```

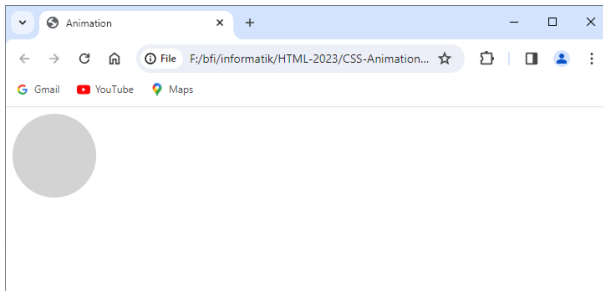


Abbildung 1: Eine Kreisfläche kann man erzeugen, wenn ein "border-radius" gewählt wird, welcher exakt der Hälfte der Höhe und Breite des Elements entspricht. Das Element muss dabei natürlich quadratisch dimensioniert werden. Dabei sind neben Höhe und Breite auch andere Eigenschaften (z.B.: Padding) zu berücksichtigen.

2. Schritt: Die CSS-Eigenschaft „animation“

Mit der CSS-Eigenschaft **animation**, die in den Regelblock des zu animierenden Elements eingefügt wird, werden mindestens die folgenden Parameter übergeben:

- Der **Name** der Animation: Dieser ist wichtig, weil durchaus verschiedene Animationseffekte in der Webseite verwendet werden können. Denkbar ist auch, mehrere Effekte für ein Element zu kombinieren. – Dieser Name kann auch mit der CSS-Eigenschaft „**animation-name**“ explizit deklariert werden.
- Die **Dauer** der Animation: Hier wird festgelegt, wie lange ein Animationsablauf dauern soll. Als Zeitvorgabe können Sekunden oder Millisekunden gewählt werden. Der Wert „0“ schaltet die Animation aus. Die Dauer der Animation kann mithilfe der CSS-Eigenschaft „**animation-duration**“ auch explizit deklariert werden.
- **Anzahl der Wiederholungen** der Animation. Wird hier „infinite“ eingetragen, wird die Animation endlos oft wiederholt. – Auch für diesen Wert gibt es eine eigene CSS-Eigenschaft: „**animation-iteration-count**“. Als Wert wird entweder „infinite“ oder ein Zahlenwert eingesetzt.

```
<style>
div {
  width: 100px;
  height: 100px;
  border-radius: 50px;
  background-color: lightgray;
  position: absolute;
  animation: farbwechsel 10s infinite;
}
</style>
```

3. Schritt: @keyframes gibt vor, was passieren soll

Aktualisiert man nun den Browser mit diesem Code, fällt jedoch auf, dass noch gar nichts passiert. Bisweilen wurden lediglich Regeln festgelegt, die den zeitlichen Verlauf einer Animation beeinflussen. Was aber genau passieren soll, beschreibt die Eigenschaft **animation** nicht!

Es müssen zusätzlich Regeln geschrieben werden, welche die Animation selbst definieren. Das passiert in den so genannten **@keyframes**.

Wer ein wenig Erfahrungen in der (prozeduralen) Programmiertechnik hat, kennt den Begriff der Funktion. Auch wenn @keyframes im Sinne einer Programmiersprache nicht als Funktion zu bezeichnen sind, ähnelt sich jedoch deren Handhabung:

- Dem „Schlüsselwort“ **@keyframes** folgt der **Name** der Animation. Dieser muss dem Namen entsprechen, der bereits in der Eigenschaft **animation** eingetragen wurde. Der Name der Animation ist also die verbindende Referenz zwischen dem zu animierenden Element und den @keyframes.
- In geschweiften Klammern – ganz im Stil der CSS-Syntax – werden nun der oder die Animationseffekt(e) beschrieben. Hierzu werden Codeblöcke **from{}** und **to{}** oder prozentuale Angaben für die Aufteilung der Zwischenschritte verwendet, in denen der Ausgangs- und der Zielzustand beschrieben wird.

In dieser ersten Aufgabe soll sich der graue Kreis in einen roten Kreis verwandeln. Grau ist die Hintergrundfarbe, so dass der „**from{}**“-Block wie folgt aussieht:

```
from {background-color: lightgrey;}
```

Der Zielzustand soll ein roter Hintergrund sein. Entsprechend wird der „**to{}**“-Block wie folgt formuliert:

```
to {background-color: red;}
```

Der vollständige Inhalt des style-Elements (der CSS-Code) sieht nun wie folgt aus. Jetzt funktioniert die Animation.

```
<style>
div {
  width: 100px;
  height: 100px;
  border-radius: 50px;
  background-color: lightgray;
  animation: farbwechsel 10s infinite;
}
@keyframes farbwechsel {
  from {background-color: lightgray;}
  to {background-color: red;}
}
</style>
```

Verschiedene Veränderungen kombinieren

Die CSS-Blöcke in den geschweiften Klammern nach „from“ und „to“ können nahezu alle CSS-Regeln¹ enthalten und damit animiert werden. Dies sind u.a.:

- Farb-Deklarationen,
- Größen-Deklarationen,
- Positionsdaten,
- Rahmen und Radius-Deklarationen,
- und verschiedene andere CSS-Regeln.

Es lassen sich auch Kombinationen dieser Regeln in nur eine Animation zusammenfassen.

¹ Eine Liste der „animierbaren“ CSS-Eigenschaften sind auf der Seite von w3schools.com zu finden:

https://www.w3schools.com/cssref/css_animatable.php

Aufgabe: Der hellgraue Kreis soll nicht nur seine Hintergrundfarbe in rot verändern, sondern auch um 250px nach rechts wandern.

Um diese Aufgabe zu erfüllen, wird der zuvor gezeigte CSS-Code im @keyframes-Block um zwei Zeilen erweitert:

```
@keyframes farbwechsel {
  from {
    background-color: lightgray;
    left: 0px;
  }
  to {
    background-color: red;
    left: 250px;
  }
}
```

Alles Andere bleibt wie bisher bestehen. Im Ergebnis ändert sich jetzt nach wie vor die Farbe der Kreisfläche, zeitgleich bewegt sich das div-Element jedoch auch nach rechts.

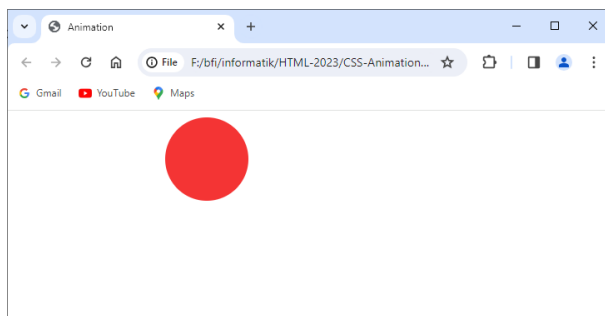


Abbildung 2: Der einst graue Kreis ändert jetzt nicht nur seine Farbe auf Rot, sondern bewegt sich im festgelegten Zeitrahmen (hier: 10 Sekunden) nach rechts auf einen Abstand von 250px zum linken Rand.

Noch ein Beispiel: Neben der Farbänderung und der Bewegung in eine Richtung soll sich die runde Fläche auch wieder zurück in ein Quadrat verwandeln.

Zur Erinnerung: Der Kreis wird durch die CSS-Eigenschaft „**border-radius**“ erzeugt, dessen Wert auf die Kantenlänge des quadratischen div-Elements abgestimmt wurde. Der Höhe und der Breite wurden die gleichen Werte zugewiesen. Wird nun der Wert für „**border-radius**“ allgemein auf 0px gesetzt, bedeutet dies, dass die Ecken des Elements ohne Rundung erscheinen. Im „to {}“-Bereich muss also „**border-radius: 0px;**“ eingetragen werden.

```
@keyframes farbwechsel {
  from {
    background-color: lightgray;
    left: 0px;
    border-radius: 50px;
  }
  to {
    background-color: red;
    left: 250px;
    border-radius: 0px;
  }
}
```

```
}  
}
```

Hinweis: Es ist nicht zwingend nötig, einen Ausgangswert im Block „`from{}`“ einzutragen, außer, dass es sich um einen klar definierten Ursprung der Animation handeln soll. Wird im Block „`from{}`“ kein Ausgangswert definiert, so wird die Animation auf Grundlage der aktuellen Formaten ausgeführt. Doch Achtung: Wenn ein Format keinen Ausgangswert – also auch nicht in den „User-Agent-Stylesheet“-Deklarationen besitzt, kann es dazu führen, dass die Animation nicht funktioniert.

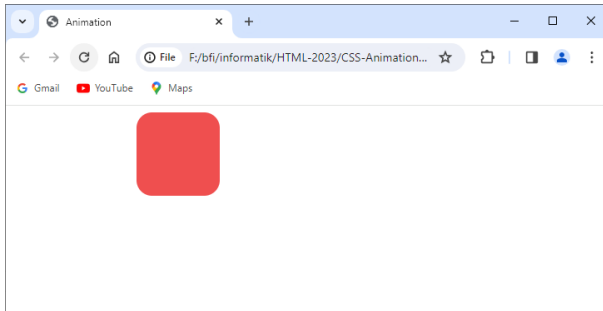


Abbildung 3: Der Zielwert für den „border-radius“ wird in der Animation neben der Positions-Änderung und dem Farbwechsel auf Null gesetzt. Die Fläche wandelt sich somit zusätzlich zum Quadrat.

Animationen in mehreren Teilschritten

Wenn in den **@keyframes** die Blöcke „`from{}`“ und „`to{}`“ verwendet werden, wird damit ein Anfangs- und ein Endpunkt der Animation festgelegt. Oft werden jedoch komplexere Animationen benötigt. Um dies zu erläutern, soll die beispielhafte Aufgabe um eine weitere Herausforderung ergänzt werden: Die sich umfärbende und zum Quadrat wandelnde Fläche soll nun einen quadratischen Weg beschreiben: Zuerst soll die Fläche um 250px nach rechts wandern und sich danach um 250px nach unten bewegen. Anschließend soll sich die Fläche wieder an den linken Rand und zum Schluss – endgültig gewandelt – wieder zum Ausgangspunkt bewegen.

Es sollen also vier Bewegungen in einer Animation durchgeführt werden. Dies ist allein mit den Blöcken **`from{}`** und **`to{}`** nicht darstellbar. Deswegen wird die Animationszeit entsprechend aufgeteilt und die einzelnen Abschnitte mit eigenen Regelblöcken belegt. In diesem Beispiel werden vor allem die Ziel-Positionen der animierten Fläche aber auch die border-radien und die Hintergrundfarbe verändert:

```
@keyframes farbwechsel {  
  0% {  
    background-color: lightgray;  
    left: 0px;  
    top: 0px;  
    border-radius: 50px;  
  }  
  25% {  
    background-color: red;  
    left: 250px;  
    top: 0px;  
    border-radius: 38px;  
  }  
}
```

```

50% {
    background-color: green;
    left: 250px;
    top: 250px;
    border-radius: 25px;}

75% {
    background-color: black;
    left: 0px;
    top: 250px;
    border-radius: 13px;
}

100% {
    background-color: gold;
    left: 0px;
    top: 0px;
    border-radius: 0px;
}
}

```

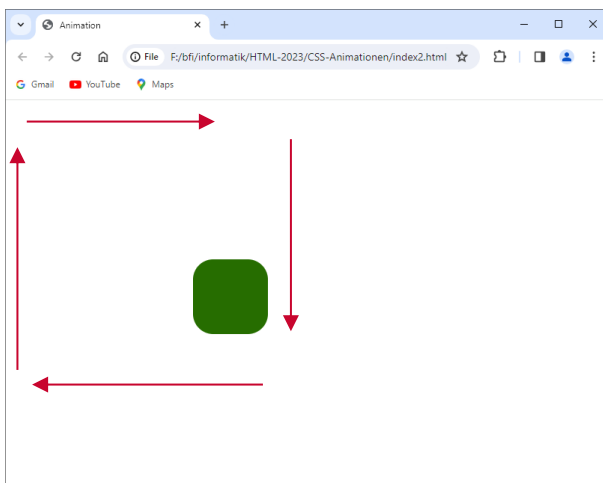


Abbildung 4: Die Animation wird Schrittweise in verschiedenen @keyframes-Regeln ausgeführt: 0% ist die Ausgangsposition, 100% die Zielposition. Dazwischen können nahezu beliebig viele Marken definiert werden, welche die Animation erreichen muss. Die zeitliche Gesamtdauer wird in der Eigenschaft „animation“ bzw. mit „animation-duration“ festgelegt.

„Richtungen“ für Animationen

Bisher wurde eine Animation in der Richtung von „from“ nach „to“ (Blöcke in den @keyframes) beschrieben. Es ist jedoch auch möglich, mithilfe der CSS-Eigenschaft „**animation-direction**“ diese Standard-Richtung zu verändern. Folgende Werte sind möglich:

- **normal** (Default-Einstellung, wenn diese Eigenschaft nicht verwendet wird): Die Animation orientiert sich an den Vorgaben des @keyframes.
- **reverse**: Entgegengesetzte Richtung zu den Deklarationen in den @keyframes
- **alternate**: Abwechselnder Lauf der Animation (zuerst in der regulären Richtung nach den Vorgaben in den @keyframes, danach in entgegengesetzter Richtung.
- **alternate-reverse**: Wie „alternate“, jedoch beginnend mit der entgegengesetzten Richtung.

- **initial und inherit:** „initial“ steht für die Standard-Vorgaben in CSS für das jeweilige Element. „inherit“ zwingt das „Erben“ der Eigenschaften aus dem vorangehenden Element.

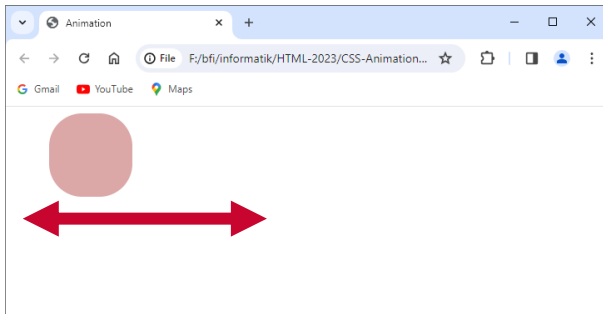


Abbildung 5: Durch die CSS-Regel "animation-direction: alternate;" wird die Animation zuerst in der lt. @keyframes vorgegebenen Richtung und dann wieder - in entgegengesetzter Ausführung - zurück zum Ausgangspunkt ausgeführt.

Zeitverzögerter Start der Animation: „animation-delay“

Möglicherweise soll die Animation nicht sofort, sondern erst mit einer gewissen Verzögerung, einem „Delay“ beginnen. Um diese Verzögerungszeit festzulegen, wird die CSS-Eigenschaft „**animation-delay**“ verwendet. Als Wert wird eine Zeit in Sekunden oder Millisekunden eingetragen.

Der folgende CSS-Code zeigt ein Beispiel:

```
div {  
    width: 100px;  
    height: 100px;  
    border-radius: 50px;  
    background-color: lightgray;  
    animation: farbwechsel 10s infinite;  
    animation-delay: 3s;  
}
```

Transition und Transform

Die CSS-Eigenschaft „**transition**“ definiert das Verhalten bei einem Übergang. So können beispielsweise die Geschwindigkeiten während eines Übergangs beeinflusst werden. Die Eigenschaft „**transform**“ bewirkt hingegen eine direkte Veränderung eines Elements. Hier findet gewissermaßen ebenfalls eine Animation statt. Beide Eigenschaften sollen nachfolgend erläutert werden.

Übergangseigenschaften mit „transition“

Mit der CSS-Eigenschaft „transition“ ist es möglich, eine Formatänderung zeitlich gleitend umzusetzen. In einem Beispiel soll die nachfolgende HTML-Überschrift h1 bei der Überfahrt mit dem Mauszeiger vom hellblauen auf einen orangefarbenen Hintergrund wechseln. Der zunächst gezeigte CSS-Code wird dies direkt umsetzen:

```
<h1>Hallo Welt!</h1>
```

Der CSS-Code für den unverzüglichen Wechsel der Hintergrundfarbe ist sehr einfach:

```
/* ursprüngliche Hintergrundfarbe hellblau */  
h1 {background-color: lightblue;}  
/* direkter Wechsel des Hintergrundes auf Orange bei Mausüberfahrt */  
h1:hover {background-color: orange;}
```

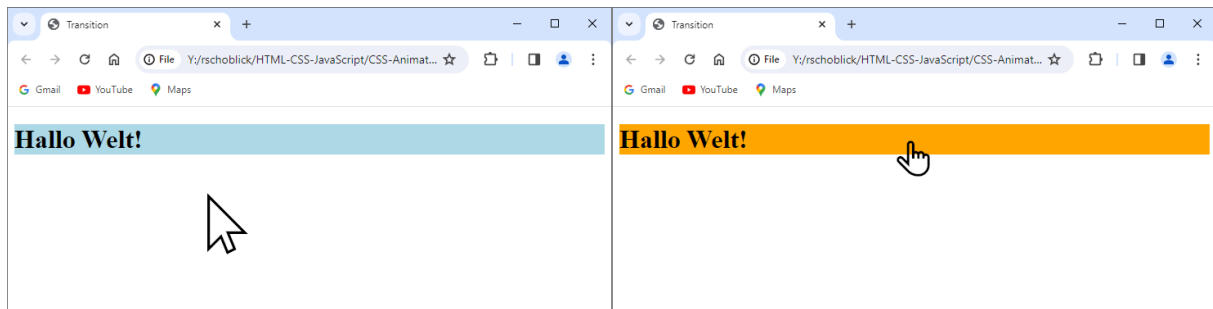


Abbildung 6: Links der Absatz mit hellblauem Hintergrund, rechts der sofortige Wechsel auf einen orangefarbenen Hintergrund bei Überfahrt mit dem Mauszeiger. Der Farbwechsel passiert augenblicklich.

Natürlich lässt sich diese Aufgabe auch – wie zuvor beschrieben – animiert lösen. Es gibt jedoch auch eine Alternative. Dabei wird der ursprüngliche CSS-Code für den „Normalzustand“ des HTML-Elements um eine weitere Zeile ergänzt:

```
h1 {
background-color: lightblue;
transition: background-color 5s;
}
```

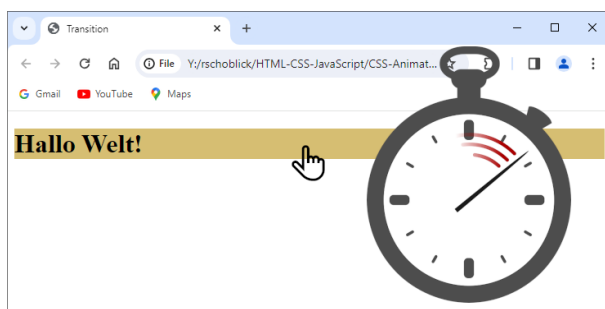


Abbildung 7: Durch die zusätzliche CSS-Eigenschaft "transition" erfolgt der Wechsel im Format nach dem Eintreten des Ereignisses (hier: :hover) innerhalb der festgelegten Zeit fließend.

Die Eigenschaft „**transition**“ benötigt mindestens zwei Werte: Die vom Übergang betroffene Eigenschaft (hier: „**background-color**“) und die Zeitdauer, welche diesem Übergang zugeschrieben wird. Wichtig ist jedoch, dass es einen **gültigen Ausgangswert** für den Übergang gibt. Solche gültigen Ausgangswerte sind auch die Standard-Einstellungen des „User-Agent-Stylesheet“, welches die Grundkonfiguration eines jeden Webbrowsers darstellt und dafür sorgt, dass die verschiedenen HTML-Elemente in Standard-Formaten erscheinen.

Problematischer ist es allerdings, wenn zum Beispiel nicht im User-Agent-Stylesheet vordefinierte Formate einen kontinuierlichen Übergang erhalten sollen. Zum Beispiel muss die Höhe bzw. die Breite eines Elements einen festen Ausgangswert erhalten, damit die Transition wirken kann.

Übergänge für verschiedene Eigenschaften eines Elements?

Im Beispiel wurde lediglich die Farbe des Hintergrundes innerhalb einer vorgegebenen Zeit variiert. Doch wie sieht die CSS-Syntax aus, wenn mehrere Eigenschaften des Elements – womöglich noch in verschiedenen Geschwindigkeiten – verändert werden sollen?

Im folgenden Beispiel soll sich deswegen nach wie vor die Hintergrundfarbe innerhalb von fünf Sekunden bei der Überfahrt der h1-Überschrift mit dem Mauszeiger wandeln. Zusätzlich soll sich aber auch die Höhe des Elements auf 200px strecken. Dafür ist für den Übergang jedoch ein längerer Zeitraum von zehn Sekunden vorgesehen.

Wie bereits erwähnt, benötigt die Transition der Höhe einen definierten Ausgangswert, der mit 50px (Höhe ohne Mauszeiger-Überfahrt) festgelegt werden soll.

Um die beiden Effekte bei der Überfahrt mit der Maus wirksam werden zu lassen, wird nur eine einzige „**transition**“-Eigenschaft in den Regelblock für das Element – hier: h1 – eingesetzt. Es wird jeweils der Name der zu ändernden Eigenschaft sowie jeweils die Zeitdauer für den Übergang durch ein Leerzeichen voneinander getrennt geschrieben. Es ergeben sich Deklarations-Paare, die jeweils durch ein Komma voneinander zu trennen sind.

```
h1 {
    background-color: lightblue;
    height: 50px;
    /* Das Komma zwischen den Deklarationen ist wichtig! */
    transition: height 10s, background-color 5s;
}
h1:hover {
    background-color: orange;
    height: 200px;
}
```

Entkoppelte Eigenschaften: transition-duration und transition property

In den vorangegangenen Abschnitten wurde die Eigenschaft „transition“ mit zwei Parametern vorgestellt: CSS-Eigenschaft („**Property**“) auf die der Übergang wirken soll und die Zeit, die der Übergang vom Anfang zum Ende benötigt („**Duration**“). Beide Werte können auch in getrennten CSS-Eigenschaften deklariert werden:

- **transition-property:**
- **transition-duration:**

Die Verwendung getrennter CSS-transition-Eigenschaften ist allerdings mit Vorsicht zu genießen, wenn die einzelnen zu verändernden Eigenschaften unterschiedliche Zeiten benötigen sollen.

Der folgende Code ist NICHT geeignet!

```
h1 {
    background-color: lightblue;
    height: 50px;
    transition-property: height;
    transition-duration: 4s;
    transition-property: background-color;
    transition-duration: 10s;
}
h1:hover {
    background-color: orange;
    height: 200px;
}
```

Es wird lediglich die zuletzt gültige Deklaration umgesetzt. Hier werden zweimal die Eigenschaften „**transition-property**“ und „**transition-duration**“ verwendet. Nur die zuletzt gültigen Regeln werden umgesetzt. Damit werden die erste transition-Eigenschaft für „height“ von der zweiten für die Hintergrundfarbe (background-color) sowie die Zeiten (4 Sekunden durch die spätere Deklaration 10 Sekunden) überschrieben. Die Höhenveränderung wird also direkt und nicht mit einem zeitlichen Übergang vollzogen.

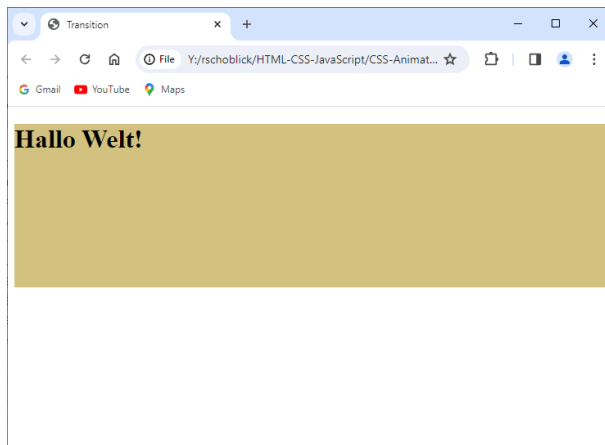


Abbildung 8: Im ersten Beispiel wird nur die Veränderung der Hintergrundfarbe zeitlich gleitend dargestellt. Die Höhenänderung des Elements erfolgt unverzüglich bei Mausüberfahrt. Da die **transition**-Regel für die Höhenänderung VOR der für die Hintergrund-Änderung gesetzt wurde, kann sie nicht wirksam werden. Sie wird überschrieben.

Besser funktioniert der nachfolgende Code. Hier werden den CSS-Eigenschaften **transition-property** und **transition-duration** verschiedene, durch ein Komma getrennte Werte übergeben. Die Reihenfolge der Parameter ist natürlich wichtig.

```
transition-property: height, background-color;
transition-duration: 4s, 10s;
```

Die Höhenänderung erfolgt nun wie gewünscht innerhalb von vier Sekunden. Die Änderung der Hintergrundfarbe benötigt zehn Sekunden.

Verzögerter Übergang: transition-delay

In den bisherigen Beispielen startete die jeweilige Animation stets sofort, sobald das triggernde Ereignis – hier: die Überfahrt mit dem Mauszeiger – eintrat. Das muss aber nicht immer gewünscht sein. So kann es von Vorteil sein, die Animation erst mit einem gewissen zeitlichen Versatz zu starten. So kann durchaus „Ruhe“ in die Darstellung einer Webseite gebracht werden, wenn der Benutzer mit dem Mauszeiger über den Bildschirm fährt. Die Animation startet dann erst, wenn der Zeiger eine gewisse Zeit über dem Element verweilt.

Ein solches Delay ist leider auch Grundlage eines beliebten „Tricks“, um Besucher einer Webseite dazu zu verleiten, einen eigentlich nicht gewünschten Link anzuklicken. So wird beispielsweise ein – durchaus interessanter – Button (z.B.: „Weiterlesen“ o.ä.) sofort eingeblendet und prominent positioniert. Nach einer kurzen Verzögerung erscheint direkt über diesen Button eine Werbebotschaft, welche direkt auf die Webseite des Werbetreibers verlinkt ist. Es wird hier die Reaktions-Trägheit des Menschen ausgenutzt, um – im harmlosesten Fall – Klickzahlen zu steigern. Über die Rechtmäßigkeit derartiger Methoden lässt sich streiten.

Das eben gezeigte Beispiel wird um die grüne Code-Zeile erweitert. Die Übergänge (Höhenänderung dauert vier und die Farbänderung zehn Sekunden) starten erst nach zwei Sekunden, wenn der Mauszeiger auf dem h1-Element liegt.

```
h1 {
    background-color: lightblue;
    height: 50px;
    transition-property: height, background-color;
    transition-duration: 4s, 10s;
    transition-delay: 2s;
}
```

```
h1:hover {  
    background-color: orange;  
    height: 200px;  
}
```

Auch hier können wieder – in der richtigen Reihenfolge durch Kommata getrennt – verschiedene Verzögerungszeiten für mehrere Übergänge programmiert werden.

Timing-Effekte für den Übergang

Mithilfe einer „**transition-timing-function**“ kann das Tempo des Übergangs beeinflusst werden (vgl. nachfolgende Tabelle). An dieser Stelle wurde bewusst auf die transition-timing-function „bezier-cube“ verzichtet. Eine Bezier-Funktion ist beispielsweise aus der „Vektorgrafik“ bekannt, wo anhand eines Anfangspunktes und eines Steuerpunktes der Verlauf einer Kurve errechnet werden kann. Ähnliches ist auch für den (zeitlichen) Verlauf des Übergangs möglich.

Eine einfache – nicht wirklich erschöpfende – Information zu dieser Zeitfunktion kann man unter

https://www.w3schools.com/cssref/func_cubic-bezier.php

finden.

Praktische Experimente sind direkt auf der folgenden Seite möglich:

https://www.w3schools.com/cssref/tryit.php?filename=trycss_func_cubic-bezier

Wert	Effekt
linear	Der Übergang erfolgt über den Zeitabschnitt im stets gleichen Tempo.
ease	Der Übergang beginnt langsam, beschleunigt im mittleren Zeitabschnitt und gleitet letztlich langsam wieder aus.
ease-in	Der Übergang beginnt langsam.
ease-out	Der Übergang endet langsam.
ease-in-out	Vergleichbar mit <i>ease</i> mit einem anderen Geschwindigkeitsprofil.

Neben den zeitlich gleitenden Übergängen können auch diskrete Schritte festgelegt werden. Auch hier soll ein Beispiel helfen, den Übergang zu verstehen:

Wieder wird ein h1-Element formatiert, welches sich bei der Überfahrt mit dem Mauszeiger verwandeln soll. Diesmal sollen verschiedene Übergänge für die Formate der Höhe und der Breite definiert werden. Es wird der folgende CSS-Code verwendet:

```
h1 {  
    background-color: lightblue;  
    width: 200px;  
    height: 50px;  
    transition-property: height, width;  
    transition-duration: 5s, 10s;  
    transition-timing-function: steps(5, start);  
}
```

```
h1:hover {  
    height: 300px;  
    width: 500px;  
}
```

In der Zeile

```
transition-timing-function: steps(5, start);
```

wird mit dem Schlüsselwort „**steps()**“ festgelegt, dass die Übergänge schrittweise und nicht wie bisher kontinuierlich erfolgen. Darüber hinaus zeigen die „blauen“ Zeilen im Code, dass zwei Eigenschaften mit verschiedenen Übergangszeiten belegt werden. Die Höhe des Elements soll sich innerhalb von 5 Sekunden, die Breite innerhalb von 10 Sekunden wandeln.

Wird dieser Code ausgeführt, so ist Folgendes zu beobachten:

- Mit der Überfahrt des Mauszeigers wird jeweils ein Teilschritt in die Breite und in die Höhe Ausgeführt.
- Mit der zweiten Sekunde ändert sich lediglich die Höhe um einen weiteren Schritt.
- Mit dem dritten („Höhenschritt“) wird der zweite Schritt in die Breite umgesetzt.
- Der vierte Höhenschritt wird wieder ohne Wirkung auf die Breite ausgeführt.
- Zeitgleich mit dem fünften Schritt bei der Höhenänderung wird der dritte Schritt in die Breite getan.
- Jetzt werden nur noch im Intervall von zwei Sekunden die beiden restlichen Schritte in die Breite umgesetzt, bis auch hier der Zielwert erreicht ist.

Die Übergangszeit wird also durch die Anzahl der „Steps“ in gleiche Teilabschnitte aufgeteilt. Dieser Zahlenwert muss ganzzahlig (Integer) und größer als Null sein.

Der zweite Parameter („**start**“ bzw. „**end**“) schreibt vor, ob die Teil-Änderung am Beginn oder am Ende des jeweiligen Intervalls erfolgen soll. Fehlt dieser Parameter, so gilt als Default-Wert „**end**“. Das eben gezeigte Beispiel hat fünf Intervalle ausgeführt, wobei die Änderungen bereits am Beginn des Intervalls umgesetzt wurden. Ändern Sie nun die folgende Zeile wie folgt und beobachten Sie, wie sich das Timing bei den Höhen- und Breitenänderungen verhält. Schreiben Sie ggfs. die Schritte wie oben beschrieben auf und stellen Sie diese Beobachtungen gegenüber.

```
transition-timing-function: steps(5, end);
```

transition-„shorthand“ für alle Eigenschaften

Zum Beginn dieses Abschnittes zur CSS-Eigenschaft „transition“ wurde bereits ein „Shorthand-Code“ aus zwei Einzeleigenschaften verwendet. Auch die Verzögerung und die Zeitfunktion können selbstverständlich in nur eine einzige CSS-Zeile gemeinsam mit anderen „**transition**“-Eigenschaften deklariert werden.

Das folgende Beispiel soll die Überschrift h1 in der folgenden Weise bei der Überfahrt mit dem Mauszeiger verändern:

- Die Veränderung soll eine Sekunde NACH dem auslösenden Ereignis (hier: Überfahrt mit dem Mauszeiger) beginnen.
- Die Veränderung soll fünf Sekunden Zeit beanspruchen (nach deren Beginn).
- Es soll lediglich die Höhe verändert werden.
- Die Veränderung soll langsam beginnen und schnell enden.

Einzelne CSS-transition-Eigenschaften, die diese Aufgabe erledigen können, sind im folgenden CSS-Code beschrieben:

```
h1 {  
    background-color: lightblue;  
    width: 200px;  
    height: 50px;  
    transition-delay: 1s;  
    transition-property: height;  
    transition-duration: 5s;  
    transition-timing-function: ease-in;  
}
```

Es werden also vier CSS-Zeilen im Code benötigt. Es geht allerdings auch kürzer in nur einer einzigen CSS-Zeile.

```
transition: 5s 1s ease-in height;
```

Die Parameter können – bis auf die Zeitvorgaben – in beliebiger Reihenfolge gesetzt werden. Bei den Zeiten wird grundsätzlich die Dauer des Übergangs (**Duration**) zuerst gesetzt. Eine eventuelle zeitliche Verzögerung des Beginns (**Delay**) muss NACH der Duration geschrieben werden!

Transformations-Effekte

Zum Abschluss dieser Kurseinheit sollen noch verschiedene Methoden vorgestellt werden, wie mithilfe von CSS Elemente skaliert und/oder gedreht werden können. Hierzu gibt es die CSS-Eigenschaften

- **scale** und
- **rotate**.

Darüber hinaus soll die recht komplexe CSS-Eigenschaft „**transformation**“ erwähnt werden.

Element drehen und skalieren

Skalierung

Zunächst einmal soll ein Element skaliert werden. Dies kann natürlich auch wieder mit einem Übergangs-Effekt (*transition*) unterstützt oder durch Animationen umgesetzt werden, was aber nicht zwingend erforderlich ist.

Wir bleiben beim Beispiel der Formatierung eines h1-Überschriften-Elements. Es soll folgender CSS-Code für die Formatierung verwendet werden.

```
h1 {  
    background-color: lightblue;  
    width: 200px;  
    height: 50px;  
    transition: scale 5s;  
}  
  
h1:hover {  
    scale: 2;  
}
```

Der Code bewirkt, dass sich die Höhe und die Breite des Elements (hier: h1) um den Faktor zwei vergrößert.

Entsprechend bewirkt der CSS-Code

```
scale: 0.5;
```

eine Verkleinerung beider Dimensionen auf die Hälfte.

Sollen die X- und Y-Richtungen (Breite und Höhe) unterschiedliche Skalierungen erfahren, so müssen zwei Parameter verwendet werden:

```
scale: 4 2;
```

Dieser Code bewirkt für das betreffende Element eine Vervielfachung der Breite und eine Verdoppelung in der Höhe.

Hinweis: Alternativ zu den absoluten Zahlenwerten für den Skalierungsfaktor können auch Prozent-Angaben verwendet werden. Eine Verkleinerung um die Hälfte der Ausgangsgröße wird mit folgender CSS-Zeile erreicht :

```
scale: 50%;
```

Rotation

CSS erlaubt es auch, die Ansicht eines Elements zu rotieren. Am anschaulichsten wird dies, wenn man das Element um die Z-Achse – die Achse, welche in den Bildschirm hinein weist – dreht. Man kann es sich sinnbildlich vorstellen, wenn man ein Blatt Papier mit einer Reißnadel an einer Pinnwand befestigt. Das Blatt kann man nun um die Nadel herum drehen. Die Drehung um die Z-Achse ist die Standard-Einstellung für die Rotate-Eigenschaft.

```
rotate: z 70deg;
```

Hinweis: Betrachtet man ein Element auf dem Bildschirm, so stellt die X-Achse die Breite und die Y-Achse die Höhe des Elements dar. Stellt man sich dieses Element 3-dimensional vor, dann entspricht die Z-Achse der Tiefe des Elements.

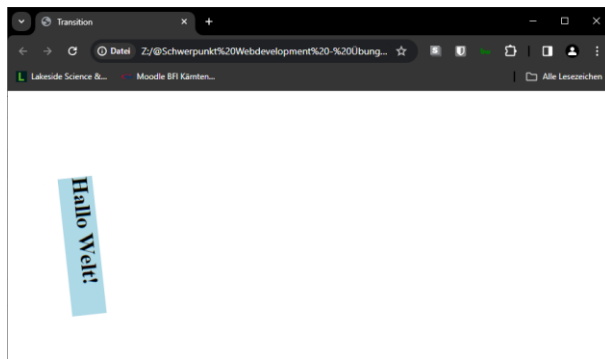


Abbildung 9: Beispiel für die Drehung um die Z-Achse. Das Element erscheint in einem vorgegebenen Winkel, ist aber stets in den vollen Abmessungen sichtbar.

Entsprechend kann das Element auch um die Hochachse (Y) gedreht werden. Optisch stellt sich dieser Effekt so dar, dass das Element immer schmaler wird. Sobald der „Nullpunkt“ (bei 90° bzw. in korrekter CSS-Schreibweise: 90deg) erreicht wird, weitet sich das Element wieder bis zur vollen Breite. Allerdings wird dessen Inhalt nun spiegelverkehrt (waagerecht) dargestellt.

```
rotate: y 70deg;
```

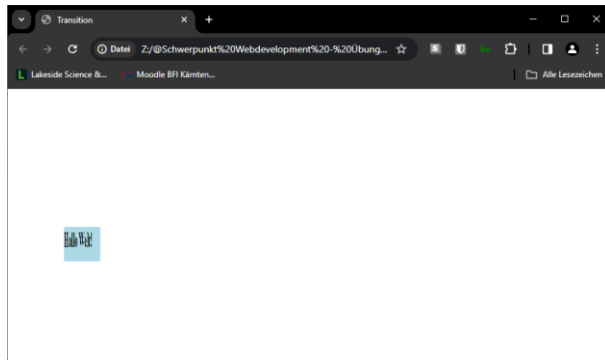


Abbildung 10: Drehung um die Hochachse (Y).

Ähnlich sieht es aus, wenn man das Element um die Querachse (X) rotieren lassen möchte: Hier wird das Element bis zu einem Winkel von 90° (90deg) in der Höhe kleiner. Bei 90deg ist das Element nicht mehr zu sehen. Winkel über 90° führen dann wieder zur Vergrößerung der Höhe des Elements, wobei dessen Inhalt jedoch auf dem Kopf steht. Ein Beispiel für eine Drehung um die x-Achse (70°):

```
rotate: x 70deg;
```

Hinweis – Interpretation der Winkelgrade: Man kann durchaus Vielfache bestimmter Winkel – zum Beispiel 720deg – verwenden, um mehrere Drehungen zu bewirken. Sinnvoll ist dies jedoch nur im Zusammenspiel mit einem animierten Effekt. Ansonsten entspricht eine Vorgabe von 405deg schlicht und einfach der Drehung um 45°.

Einheit	Bedeutung
deg	Winkel in Grad (eine volle Umdrehung = 360deg)
rad	Winkel in Grad-Radian ($2\pi \sim 6.28\text{rad}$ entsprechen eine volle Umdrehung)
turn	Anzahl der vollen Umdrehungen

Auch die Drehung um mehrere Achsen ist möglich. Wird diese obendrein animiert (**animation/@keyframes** oder durch einen Übergang), dann lassen sich sehr spektakuläre Effekte erzielen.

Drehungen über alle drei Achsen lassen sich vektoriell erzeugen. Hierbei werden zuerst die drei Achsen-Vektoren (x, y und z) durch Werte von jeweils 0 bis 1 und anschließend der bereits bekannte Rotationswinkel (in deg oder rad) bzw. die Zahl der Umdrehungen (**turn**) festgelegt.

Eine Drehung um jeweils 45° auf der Y- und Z-Achse wird demnach wie folgt deklariert:

```
rotate 0 1 1 45deg;
```

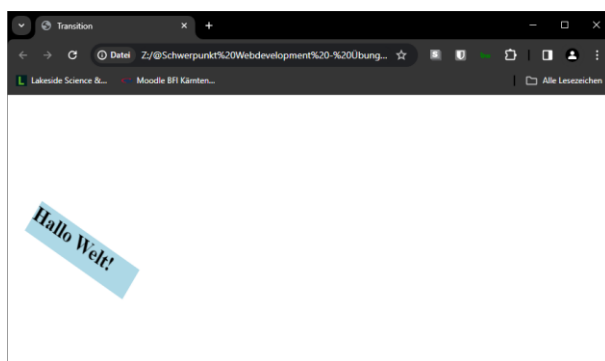


Abbildung 11: Ergebnis der Drehung eines Elements auf der Y- und Z-Achse um jeweils 45°

Hinweis: Die CSS-Eigenschaften **rotate** und **scale** sind per se keine Animationen! Es sind jedoch animierbare Eigenschaften. Sie können statisch eingesetzt werden – beispielsweise um ein Element in

einem „schrägen Winkel“ fest anzuordnen oder über eine Animation beispielsweise ein Element dauerhaft in Drehung zu versetzen.

3D-Effekte mit der CSS-Eigenschaft „transform“

Abschließend soll noch auf die CSS-Eigenschaft „transform“ hingewiesen werden. Auch diese ist animierbar (**animation/@keyframes** sowie **transition**) und bewirkt Änderungen eines Elements mit 3D-Effekten.

Neben der Drehung (**rotate**) und der Größen-Skalierung (**scale**) können Elemente auch schräg dargestellt (**skew**) oder in ihrer Position verändert (**translate**) werden.

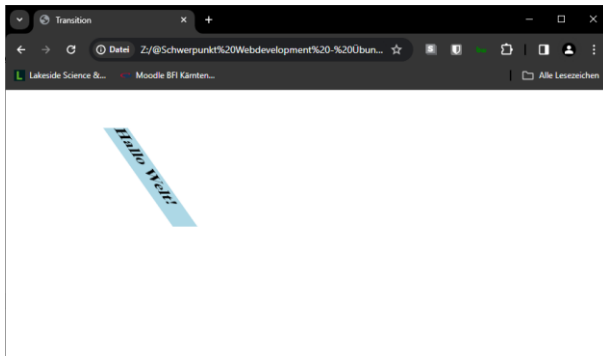


Abbildung 12: Beispiel für **transform: rotate3D(1,0,1,1.57rad);**

transform: rotate()

Ähnlich wie bei der CSS-Eigenschaft „rotate“ lassen sich Dreheffekte durch die Eigenschaft „transform“ erzeugen. Auch hier können die Drehungen an der X-, der Y- und an der Z-Achse erfolgen. Der Winkel wird in gewohnter Weise (mit den Einheiten deg oder rad) angegeben.

Beispiele für **transform: rotate()** liefert die folgende Tabelle.

CSS-Code	Wirkung
<code>transform: rotate(Winkel);</code>	Drehung um die X-Achse
<code>transform: rotateX(Winkel);</code>	Drehung um die X-Achse
<code>transform: rotateY(Winkel);</code>	Drehung um die Y-Achse
<code>transform: rotateZ(Winkel);</code>	Drehung um die Z-Achse
<code>transform: rotate3D(x, y, z, Winkel);</code>	3D-Drehung (Achsen vektorielle Faktoren)

transform: scale()

Neben der grundlegenden Festlegung der Höhe (**height**) und Breite (**width**) sowie der Skalierung mithilfe der Eigenschaft **scale** können Skalierungen auch mithilfe der Transformation ausgehend von der aktuellen Größe des Elements vorgenommen werden. Echte 3D-Skalierungen sind jedoch in der Regel im Webdesign eher theoretischer Natur.

CSS-Code	Wirkung
<code>transform: scale(FaktorX, FaktorY);</code>	2D-Skalierung in X- und Y-Richtung
<code>transform: scaleX(Faktor);</code>	Skalierung entlang der X-Achse
<code>transform: scaleY(Faktor);</code>	Skalierung entlang der Y-Achse
<code>transform: scaleZ(Faktor);</code>	Skalierung entlang der Z-Achse
<code>transform: scale3D(x, y, z);</code>	3D-Skalierung: Faktoren für die Achsen (nur sinnvoll bei 3D-Effekten)

Die Angaben der Faktoren sind in der Form reiner Zahlenwerte (Dezimalstellen durch einen Punkt getrennt) oder in Prozent (%) möglich.

transform: skew()

„**skew**“ steht für Schräg- oder Schiefele. **skew()** wird als Wert der transform-Eigenschaft übergeben. Auch hier werden Winkelangaben in deg oder rad als Parameter in den Klammerausdruck übergeben. Definiert sind **skew()**-Varianten für die X- und die Y-Achse. Je nachdem, ob der Winkelwert positiv oder negativ gesetzt wird, wird die Schräglage nach links oder rechts ausgeführt.

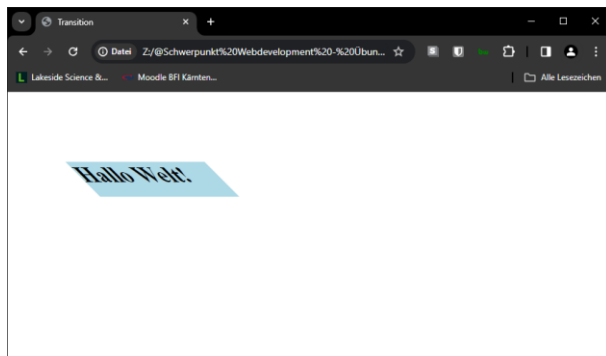


Abbildung 13: Schräglage mit einem Winkel von +45°

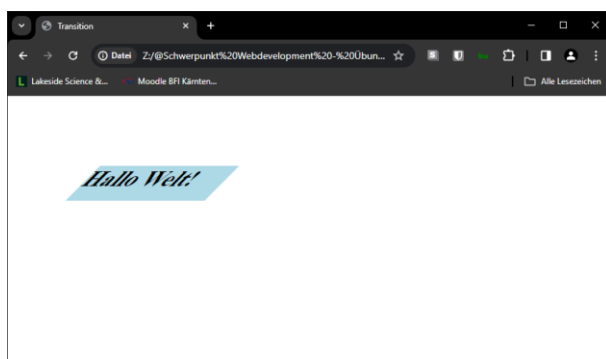


Abbildung 14: Schräglage mit einem Winkel von -45°

transform: translate()

Die CSS-Regel „**transform: translate(x, y);**“ kann insbesondere für Animationen sehr sinnvoll eingesetzt werden, weil das Objekt relativ in X- und/oder Y-Richtung verschoben werden kann. Ausgangspunkt der Verschiebung ist stets die aktuelle Position des Elements.

Das folgende Beispiel zeigt wieder ein Überschriften-Element h1, welches in einer Animation den Weg eines Quadrates beschreiben soll. Ein ähnliches Beispiel wurde bereits ab Seite 7 beschrieben, jedoch waren die CSS-Codes in den **@keyframes** für einen vergleichbaren Effekt umfangreicher.

```
h1 {
...
position: absolute;
left: 200px;
top: 200px;
animation: trans 20s infinite;
}
```

```
@keyframes trans {  
  25% {  
    transform: translate(400px, 0px);  
  }  
  50% {  
    transform: translate(400px, 400px);  
  }  
  75% {  
    transform: translate(0px, 400px);  
  }  
  100% {  
    transform: translate(0px, 0px);  
  }  
}
```

Während im Beispiel auf der Seite 7 zwei CSS-Eigenschaften für die „top-“ und „left-“Position festgelegt wurden, brauchte es in Beispiel mit „**transform: translate()**“ nur jeweils eine Zeile, in der die relativen Verschiebungen des Elements festgelegt werden. Ein weiterer Unterschied in den Beispielen stellt die Flexibilität dar. Mit „**transform: translate()**“ kann die Ausgangsposition an einer beliebigen Stelle in der Seite platziert werden. Im Beispiel ab Seite 7 muss beachtet werden, welches Element den Nullpunkt für die Koordinaten des zu animierenden Elements setzt.